

Problem 1: Vectors & Planes Review

(a)

Provide a formula for computing the angle between two nonzero d -dimensional vectors $\bar{x}^{(1)}$ and $\bar{x}^{(2)}$. Now suppose $\bar{x}^{(1)} = [-\frac{\sqrt{3}}{4}, \frac{1}{4}]^T$ and $\bar{x}^{(2)} = [\frac{\sqrt{3}}{2}, \frac{1}{2}]^T$. What is the magnitude of each vector, and what is the angle between them in degrees?

$$\bar{x}^{(1)} \cdot \bar{x}^{(2)} = \|\bar{x}^{(1)}\| \|\bar{x}^{(2)}\| \cos \theta$$

$$\cos \theta = \frac{\bar{x}^{(1)} \cdot \bar{x}^{(2)}}{\|\bar{x}^{(1)}\| \|\bar{x}^{(2)}\|}$$

$$\theta = \arccos \left(\frac{\bar{x}^{(1)} \cdot \bar{x}^{(2)}}{\|\bar{x}^{(1)}\| \|\bar{x}^{(2)}\|} \right)$$

$$\|\bar{x}^{(1)}\| = \sqrt{\left(-\frac{\sqrt{3}}{4}\right)^2 + \frac{1}{4^2}} = \sqrt{\frac{1}{4}} = \frac{1}{2}$$

$$\|\bar{x}^{(2)}\| = \sqrt{\left(\frac{\sqrt{3}}{2}\right)^2 + \frac{1}{2^2}} = 1$$

$$\theta = \arccos \left(\frac{-\frac{1}{4}}{\frac{1}{2} \cdot 1} \right) = \arccos \left(-\frac{1}{2} \right) = 120^\circ$$

(b)

Fix a d -dimensional vector $\bar{\theta} = [\theta_1, \dots, \theta_d]^T$ and $\bar{\theta} \neq 0$. Let V be the hyperplane in $\mathbb{R}_d$ that includes all points $\bar{x} = [x_1, \dots, x_d]^T$ s.t. $\bar{x} \cdot \bar{\theta} = 0$. Find all of the d -dimensional vectors that are orthogonal to V and have a length of C . Justify your answer.

Let S_C be the set of all d -dimensional vectors that are orthogonal to V and have a length of C .

Consider $S = \left\{ \pm \frac{C}{\|\bar{\theta}\|} \bar{\theta} \right\}$

Let $\bar{w} \in S$ and $\bar{v} \in V$.

$$\bar{w} \cdot \bar{v} = \pm \frac{C}{\|\bar{\theta}\|} \bar{\theta} \cdot \bar{v} = 0$$

and

$$\|\bar{w}\| = \sqrt{\sum_{i=1}^d \left(\pm \frac{C}{\|\bar{\theta}\|} \theta_i \right)^2} = \frac{C}{\|\bar{\theta}\|} \|\theta\| = C$$

Thus, $\bar{w} \in S_C$, so $S \subseteq S_C$

Let $\bar{w} \in S_C$, $\lambda = \frac{\bar{w} \cdot \bar{\theta}}{\|\bar{\theta}\|^2}$, and $\bar{u} = \bar{w} - \lambda \bar{\theta}$.

$$\bar{u} \cdot \bar{\theta} = (\bar{w} - \lambda \bar{\theta}) \cdot \bar{\theta} = \bar{w} \cdot \bar{\theta} - \lambda (\bar{\theta} \cdot \bar{\theta}) = \bar{w} \cdot \bar{\theta} - \bar{w} \cdot \bar{\theta} = 0$$

Thus, $\bar{u} \in V$, and since $\bar{w} \in S_C$ this implies $\bar{u} \cdot \bar{w} = 0$.

$$0 = \bar{u} \cdot \bar{w} = (\lambda \bar{\theta} + \bar{u}) \cdot \bar{u} = \lambda (\bar{\theta} \cdot \bar{u}) + (\bar{u} \cdot \bar{u}) = (\bar{u} \cdot \bar{u}) = \|\bar{u}\|^2$$

$$\|\bar{u}\| = 0 \implies \bar{u} = 0$$

Thus, $\bar{w} = \lambda \bar{\theta}$, and since $\|\bar{w}\| = C$

$$\begin{aligned} C = \|\bar{w}\| &= \|\lambda \bar{\theta}\| \\ &= |\lambda| \|\bar{\theta}\| \\ |\lambda| &= \frac{C}{\|\bar{\theta}\|} \lambda = \pm \frac{C}{\|\bar{\theta}\|} \end{aligned}$$

Thus, $\bar{w} \in S$, so $S_C \subseteq S$.

Therefore, $S = S_C$.

(c)

Consider an arbitrary d -dimensional point $\bar{x} \in \mathbb{R}^d$ and a hyperplane through the origin parameterized by $\bar{\theta} = [\theta_1, \dots, \theta_d]^T$. The signed distance of $\bar{x}$ from the plane is the perpendicular distance between $\bar{x}$ and the hyperplane (distance between $\bar{x}$ and its projection on the hyperplane), multiplied by +1 if $\bar{x}$ lies on the same side of the plane as the vector $\bar{\theta}$ points and by -1 if $\bar{x}$ lies on the opposite side.

i.

Derive the expression for the signed distance of $\bar{x}$ from the hyperplane.

From linear algebra we can write $\bar{x} \in \mathbb{R}_d$ uniquely in orthogonal components,

$$\bar{x} = \bar{x}_{\parallel} + \bar{x}_{\perp}$$

where $\bar{x}_{\parallel}$ is parallel to $\bar{\theta}$ and $\bar{x}_{\perp}$ is orthogonal to $\bar{\theta}$.

Since $\bar{x}_{\parallel}$ is parallel to $\bar{\theta}$

$$\bar{x}_{\parallel} = \lambda \bar{\theta}$$

s.t. $\lambda \in \mathbb{R}_d$.

$$\begin{aligned} \bar{\theta} \cdot \bar{x} &= \bar{\theta} \cdot (\bar{x}_{\parallel} + \bar{x}_{\perp}) = \bar{\theta} \cdot \bar{x}_{\parallel} + \bar{\theta} \cdot \bar{x}_{\perp} = \bar{\theta} \cdot \bar{x}_{\parallel} \\ &= \lambda \bar{\theta} \cdot \bar{\theta} \\ &= \lambda \|\bar{\theta}\|^2 \\ \lambda &= \frac{\bar{\theta} \cdot \bar{x}}{\|\bar{\theta}\|^2} \end{aligned}$$

Therefore, $\|\bar{x}_{\parallel}\| = \frac{|\bar{x} \cdot \bar{\theta}|}{\|\bar{\theta}\|}$ is the distance of $\bar{x}$ from the hyperplane.

But we wanted the signed distance, but since of $\bar{x} \cdot \bar{\theta} > 0$ when $\bar{x}$ and $\bar{\theta}$ are on the same side and negative when on opposite sides, we can just remove the absolute value bar.

Thus, the signed distance is

$$\frac{\bar{\theta} \cdot \bar{x}}{\|\bar{\theta}\|}$$

ii.

Now consider a hyperplane with offset (i.e. $\bar{\theta} \cdot \bar{x} + b = 0$). How does the solution from part (i) change?

Let $\bar{x}_0$ be on the hyperplane. Then we can do the same thing as in (i) but treat $\bar{x}_0$ as the origin, so we look at the vector $\bar{x} - \bar{x}_0$.

Thus, from (i) we get the signed distance is

$$\frac{\bar{\theta} \cdot \bar{x} - \bar{x}_0}{\|\bar{\theta}\|} = \frac{\bar{\theta} \cdot \bar{x} - \bar{\theta} \cdot \bar{x}_0}{\|\bar{\theta}\|} = \frac{\bar{\theta} \cdot \bar{x} + b}{\|\bar{\theta}\|}$$

iii.

Let p be the plane consisting of the set of points for which $-2x_1 + \sqrt{5}x_2 + 3 = 0$.

i. $\frac{-2*6 + \sqrt{5}*7 + 3}{\sqrt{4+5}} = \frac{-9+7\sqrt{5}}{3}$

ii. $\frac{-2*0 + \sqrt{5}*0 + 3}{3} = 1$

Problem 2: Feature Vectors

Consider a sequence of n -dimensional data points, $\bar{x}^{(1)}, \bar{x}^{(2)}, \dots$, and a sequence of m -dimensional feature vectors, $\bar{z}^{(1)}, \bar{z}^{(2)}, \dots$, extracted from the $\bar{x}$'s by a linear transformation, $\bar{z}^{(i)} = A\bar{x}^{(i)}$.

(a)

Suppose $n = 6$, $m = 2$, with z_1 defined as average of all the elements of $\bar{x}$, and z_2 defined as the average of the first and second elements of $\bar{x}$ minus the average of the third through sixth elements of $\bar{x}$. Determine A .

$$A = \begin{bmatrix} 1/6 & 1/6 & 1/6 & 1/6 & 1/6 & 1/6 \\ 1/2 & 1/2 & -1/4 & -1/4 & -1/4 & -1/4 \end{bmatrix}$$

$$A\bar{x} = \begin{bmatrix} 1/6 & 1/6 & 1/6 & 1/6 & 1/6 & 1/6 \\ 1/2 & 1/2 & -1/4 & -1/4 & -1/4 & -1/4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{bmatrix} = \begin{bmatrix} \frac{x_1+x_2+x_3+x_4+x_5+x_6}{6} \\ \frac{x_1+x_2}{2} - \frac{x_3+x_4+x_5+x_6}{4} \end{bmatrix}$$

(b)

Assume $m < n$, suppose that $h(\bar{z}, \bar{\theta}_z) = \text{sign}(\bar{\theta}_z \cdot \bar{z})$ is a classifier for the feature vectors, and $h(\bar{x}, \bar{\theta}_x) = \text{sign}(\bar{\theta}_x \cdot \bar{x})$ is a classifier for the data vectors. Given a $\bar{\theta}_z$ that produces good classifications of the feature vectors, is there a $\bar{\theta}_x$ that will identically classify the associated $\bar{x}$'s? If so, what's the relationship between $\bar{\theta}_x$ and $\bar{\theta}_z$?

Yes there is,

$$\bar{\theta}_z \cdot z = \bar{\theta}_z^T (A\bar{x}) = (\bar{\theta}_z^T A)\bar{x} = (A^T \bar{\theta}_z)^T \bar{x} = A^T \bar{\theta}_z \cdot \bar{x}$$

and the relationship is

$$\bar{\theta}_x = A^T \bar{\theta}_z$$

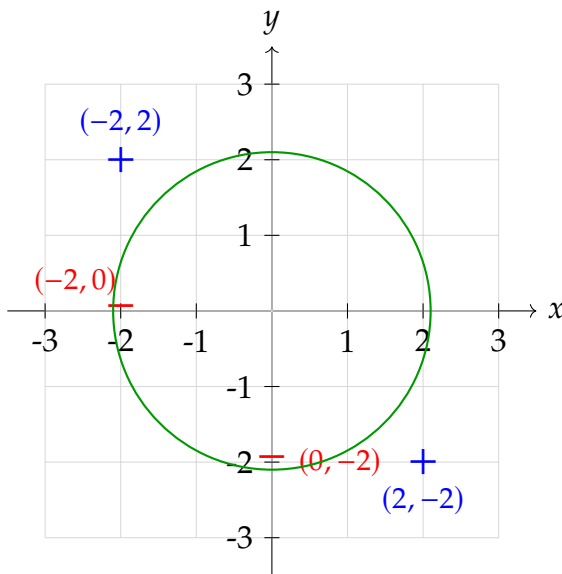
Problem 3: Decision Boundaries

For each of the following parameterized families of classifiers, give an example (find the parameters) of a family member that can correctly classify the above data, or explain why no such family member exists. Make sure to specify the direction of the negative & positive decision.

(a)

Inside or outside a circle centered at the origin with radius r .

Yes, let $r = 2.1$ with negative on the inside.



(b)

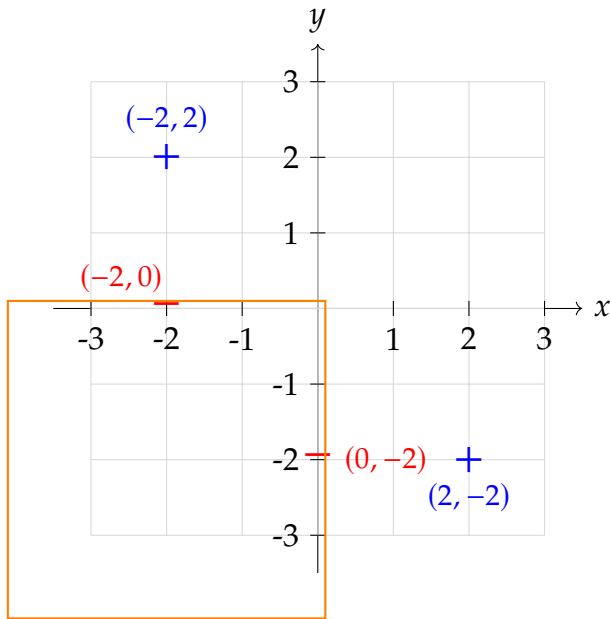
Above or below a line through the origin with normal (a, b) .

This is not possible since both positive points are on a line through the origin. That means if you have any line through the origin either positive points are on either side of the line, or they are on the line.

(c)

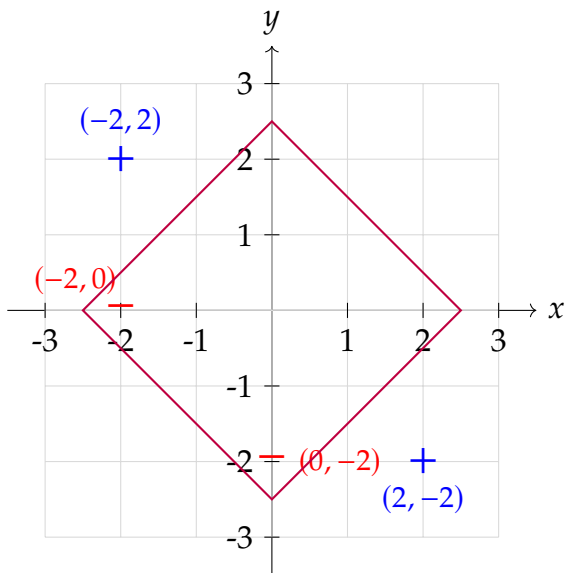
Inside or outside a square centered at (a, b) with sides parallel to the axes and side length s .

Yes, let the center be $(-2, -2)$ with $s = 4.2$. Positive is on the outside.

**(d)**

Inside or outside a square centered at the origin with side length s and counter-clockwise rotation β , where $\beta = 0$ implies sides parallel to the axes.

Yes rotate 90° and let $s = 2.5\sqrt{2}$ with the positive on the outside.



Problem 4: Perceptron Algorithm with Offset

Consider a sequence of 2-dimensional data points, $\bar{x}^{(1)}, \bar{x}^{(2)}, \dots, \bar{x}^{(n)}$ and their corresponding labels $y^{(1)}, y^{(2)}, \dots, y^{(n)}$. Recall the perceptron algorithm updates the parameters whenever $y^{(i)} \neq h(\bar{x}^{(i)}; \bar{\theta})$ where $h(\bar{x}^{(i)}; \bar{\theta}) = \text{sign}(\bar{\theta} \cdot \bar{x}^{(i)} + b)$. Assume that the points are linearly separable, and that both $\bar{\theta}$ and b are initialized to zero. Let α_i denote the number of times $\bar{x}^{(i)}$ is misclassified during training.

(a)

Derive the final decision boundary for the perceptron in terms of α_i , $\bar{x}^{(i)}$ and $y^{(i)}$.

$$\bar{\theta} = \sum_{i=1}^n \alpha_i y^{(i)} \bar{x}^{(i)}$$

$$b = \sum_{i=1}^n \alpha_i \cdot y^{(i)}$$

(b)

$\bar{\theta} = [-8, 4]$, and $b = -11$

i	Number of Misclassifications
0	6
1	19
2	0
3	2
4	12
5	0

```
def all_correct(X: np.array, y: np.array, theta: np.array, b: float):
    """
```

Args:

X: np.array, shape (n, d)

y: np.array, shape (n,)

theta: np.array, shape (d,), normal vector of decision boundary

b: float, offset

```
Returns true if the linear classifier specified by theta and b  
↪ correctly classifies all examples  
"""
```

```
for Xi, yi in zip(X,y):  
    if yi * (theta @ Xi + b) <= 0:  
        return False  
return True
```

```
def perceptron(X: np.array, y: np.array):
```

```
    """
```

```
Implements the Perceptron algorithm for binary linear classification.
```

```
Args:
```

```
    X: np.array, shape (n, d)
```

```
    y: np.array, shape (n,)
```

```
Returns:
```

```
    theta: np.array, shape (d,)
```

```
    b: float
```

```
    alpha: np.array, shape (n,).
```

```
        Misclassification vector, in which the i-th element is has the
```

```
        ↪ number of times
```

```
        the i-th point has been misclassified)
```

```
    """
```

```
n, d = X.shape
```

```
theta = np.zeros((d,))
```

```
b = 0
```

```
alpha = np.zeros((n,))
```

```
while not all_correct(X,y,theta,b):
```

```
    for i,(Xi,yi) in enumerate(zip(X,y)):
```

```
        if yi * (theta @ Xi + b) <= 0:
```

```
            theta += yi * Xi
```

```
            b += yi
```

```
            alpha[i]+=1
```

```
return theta, b, alpha
```

(c)

Given a set of linearly separable points, does the order in which the points are presented to the algorithm affect whether or not the algorithm will converge?

No, convergence depends on linear separability of the data, which is independent of the order points are presented.

(d)

In general, assuming the algorithm converges, could changing the order in which data points are considered change the solution to which the algorithm converges

Yes, different orderings produce different update sequences and can converge to different separating hyperplanes.

Problem 5: Perceptron Convergence Proof

(a)

Given a separating hyperplane parameterized by $\bar{\theta}^{*'} \in \mathbb{R}^d$, $\|\bar{\theta}^{*'}\| \neq 0$, prove that we can parameterize this same hyperplane by $\bar{\theta}^*$ s.t. $\|\bar{\theta}^*\| = 1$.

Proof.

Let

$$\bar{\theta}^* = \frac{\bar{\theta}^{*'}}{\|\bar{\theta}^{*'}\|}$$

$$\|\bar{\theta}^*\| = \frac{\|\bar{\theta}^{*'}\|}{\|\bar{\theta}^{*'}\|} = 1$$

Let $\bar{x}$ be on the hyperplane described by $\bar{\theta}^{*'}$. Consider,

$$\bar{\theta}^* \cdot \bar{x} = \|\bar{\theta}^{*'}\|(\bar{\theta}^{*' \cdot \bar{x}}) = 0$$

and by similar argument there will be no extra vectors that will be zero since if $\bar{\theta}^{*' \cdot \bar{x}} \neq 0$ then $\bar{\theta}^* \cdot \bar{x} \neq 0$.

Thus, $\bar{\theta}^*$ parameterizes the same hyperplane. ■

(b)

Now, prove that we can rescale all of our data points so that $\|\bar{x}^{(i)}\| \leq 1, \forall i = 1, \dots, n$ without changing the classification of any of the points with respect to the hyperplane defined by $\bar{\theta}^*$.

Proof.

Let

$$\bar{x}^{(i)*} = \frac{\bar{x}^{(i)}}{\|\bar{x}^{(i)}\|}$$

for all $i \in \{1, \dots, n\}$

$$\|\bar{x}^{(i)*}\| = 1 \leq 1$$

If $\bar{y}^{(i)} \bar{\theta}^{*' \cdot \bar{x}^{(i)}} > 0$ then

$$\bar{y}^{(i)} \bar{\theta}^* \cdot \bar{x}^{(i)*} = \|\bar{\theta}^{*'}\| \cdot \|\bar{x}^{(i)}\| (\bar{\theta}^{*' \cdot \bar{x}^{(i)}}) > 0$$

and if $\bar{y}^{(i)} \bar{\theta}^{*'} \cdot \bar{x}^{(i)} < 0$ then

$$\bar{y}^{(i)} \bar{\theta}^* \cdot \bar{x}^{(i)*} < 0$$

by the same argument.

Therefore, since the dot products are the same sign as the originals this implies that the classification is the same. ■

(c)

Let us start with proving a lower bound on the magnitude of $\bar{\theta}^{(k)}$.

i.

Prove that $\bar{\theta}^{(k+1)} \cdot \bar{\theta}^* \geq \bar{\theta}^{(k)} \cdot \bar{\theta}^* + \gamma$.

Proof.

Let $\bar{x}^{(i)}$ be the vector that updated $\theta^{(k)}$ to $\theta^{(k+1)}$. Then

$$\theta^{(k+1)} = \theta^{(k)} + y^{(i)} \bar{x}^{(i)}$$

so

$$\begin{aligned} \bar{\theta}^{(k+1)} \cdot \bar{\theta}^* &= (\theta^{(k)} + y^{(i)} \bar{x}^{(i)}) \cdot \bar{\theta}^* \\ &= \theta^{(k)} \cdot \bar{\theta}^* + y^{(i)} (\bar{x}^{(i)} \cdot \bar{\theta}^*) \\ &\geq \theta^{(k)} \cdot \bar{\theta}^* + \gamma \end{aligned} \quad \text{This is because } \gamma \text{ is the min of all the } y^{(i)} (\bar{x}^{(i)} \cdot \bar{\theta}^*).$$

ii.

Prove that $\|\bar{\theta}^{(k+1)}\| \geq (k+1)\gamma$.

Proof.

From *i.* we have that $\bar{\theta}^{(k+1)} \cdot \bar{\theta}^* \geq \bar{\theta}^{(k)} \cdot \bar{\theta}^* + \gamma$. Applying this formula recursively we get

$$\bar{\theta}^{(k+1)} \cdot \bar{\theta}^* \geq (k+1)\gamma$$

since $\theta^{(0)} \cdot \bar{\theta}^* = 0$ for the last step. Thus,

$$\begin{aligned} \bar{\theta}^{(k+1)} \cdot \bar{\theta}^* &\geq (k+1)\gamma \\ \|\bar{\theta}^{(k+1)}\| \cdot \|\bar{\theta}^*\| \cos(\alpha) &\geq (k+1)\gamma \quad \text{Where } \alpha \text{ is the angle between } \bar{\theta}^{(k+1)} \text{ and } \bar{\theta}^*. \end{aligned}$$

Since $|\cos(\alpha)| \leq 1$ and $\|\bar{\theta}^*\| = 1$ this implies

$$\begin{aligned}\|\bar{\theta}^{(k+1)}\| &\geq \|\bar{\theta}^{(k+1)}\| \cdot \|\bar{\theta}^*\| \cos(\alpha) \geq (k+1)\gamma \\ \|\bar{\theta}^{(k+1)}\| &\geq (k+1)\gamma\end{aligned}$$

(d)

Now we will prove an upper bound on the magnitude of $\bar{\theta}^{(k)}$.

i.

Prove that $\|\bar{\theta}^{(k+1)}\|^2 \leq \|\bar{\theta}^{(k)}\| + 1$.

Proof.

Let $\bar{x}^{(i)}$ be the vector that updated $\theta^{(k)}$ to $\theta^{(k+1)}$. Then

$$\begin{aligned}\|\bar{\theta}^{(k+1)}\|^2 &= \bar{\theta}^{(k+1)} \cdot \bar{\theta}^{(k+1)} \\ &= (\bar{\theta}^{(k)} + y^{(i)} \bar{x}^{(i)}) \cdot (\bar{\theta}^{(k)} + y^{(i)} \bar{x}^{(i)}) \\ &= \|\bar{\theta}^{(k)}\|^2 + \|\bar{x}^{(i)}\|^2 + 2y^{(i)}(\bar{\theta}^{(k)} \cdot \bar{x}^{(i)})\end{aligned}$$

Since $\|\bar{x}^{(i)}\| \leq 1$

$$\begin{aligned}&\leq \|\bar{\theta}^{(k)}\|^2 + 1 + 2y^{(i)}(\bar{\theta}^{(k)} \cdot \bar{x}^{(i)}) \\ &\leq \|\bar{\theta}^{(k)}\|^2 + 1\end{aligned}$$

Since $y^{(i)}(\bar{\theta}^{(k)} \cdot \bar{x}^{(i)}) \leq 0$

Therefore, $\|\bar{\theta}^{(k+1)}\|^2 \leq \|\bar{\theta}^{(k)}\| + 1$.

ii.

Now Prove that $\|\bar{\theta}^{(k+1)}\|^2 \leq k + 1$

Proof.

Again applying the formula recursively we get $\|\bar{\theta}^{(k+1)}\|^2 \leq (k+1)$
since $\|\theta^{(0)}\| = 0$ for the last step.

(e)

Relate inequalities in (c)(ii) and (d)(ii) to prove an upper bound on $k + 1$ solely in terms of γ .

Proof.

From (c)(ii) we get

$$(k + 1)\gamma \leq \|\bar{\theta}^{(k+1)}\|$$

From (d)(ii) we get

$$(k + 1)^2\gamma^2 \leq \|\bar{\theta}^{(k+1)}\|^2 \leq k + 1$$

$$(k + 1)^2\gamma^2 \leq k + 1$$

$$k + 1 \leq \frac{1}{\gamma^2}$$



Problem 6: Linear Regression - Optimization Methods

(a)

You will now implement and use three different optimization methods to find the coefficients θ_1 and θ_0 (slope and intercept, respectively) that minimize the squared loss for a model of the form $\hat{y} = \theta_1 x + \theta_0$. Include screenshots of your code for each function.

i.

- Implement `ls_gradient_descent(X,y)`.
- Implement `ls_stochastic_gradient_descent(X,y)`.
- Implement `ls_closed_form_optimization(X,y)`.

```
def ls_gradient_descent(X, y, learning_rate=0):  
    """  
    Implement the Gradient Descent (GD) algorithm for least squares  
    ↪ regression.  
    Note:  
    - Please use the following stopping criteria together: number  
    ↪ of iterations >= 1e6 or |new_loss - prev_loss| <= 1e-10  
  
    Args:  
        X: np.array, shape (n, d)  
        y: np.array, shape (n,)   
  
    Returns:  
        theta: np.array, shape (d,)   
    """  
    n, d = X.shape  
    theta = np.zeros(d)  
  
    eps = 1e-10  
    max_iter = 1e6  
    n_iter = 0
```

```

step = learning_rate
prev_loss = np.inf
new_loss = calculate_squared_loss(X, y, theta)

while n_iter < max_iter and abs(new_loss - prev_loss) > eps: #
    ↪ Implement the correct stopping criteria
    n_iter += 1
    grad = []

    for xt, yt in zip(X, y):
        # Append the gradient of the loss function evaluated at
        ↪ each point
        grad.append(-(yt - theta @ xt) * xt)

    grad = np.mean(grad, axis=0)
    theta = theta - (step * grad)

    prev_loss = new_loss
    new_loss = calculate_squared_loss(X, y, theta)

print("Learning rate:", learning_rate, "\t\t\tNum iterations:",
    ↪ n_iter)
return theta

```

```

def ls_stochastic_gradient_descent(X, y, learning_rate=0):
    """
    Implement the Stochastic Gradient Descent (SGD) algorithm for least
    ↪ squares regression.
    Note:
        - Please do not shuffle your data points.
        - Please use the following stopping criteria together: number
        ↪ of iterations  $\geq 1e6$  or  $|new\_loss - prev\_loss| \leq 1e-10$ 

    Args:
        X: np.array, shape (n, d)
        y: np.array, shape (n,)
        learning_rate: the learning rate for the algorithm
    """

```

```
    Returns:
        theta: np.array, shape (d,)
    """
    n, d = X.shape
    theta = np.zeros(d)
    adaptive = (learning_rate == 'adaptive')

    eps = 1e-10
    max_iter = 1e6
    n_iter = 0
    epochs = 0

    step = learning_rate
    prev_loss = np.inf
    new_loss = calculate_squared_loss(X, y, theta)

    while n_iter < max_iter and abs(new_loss - prev_loss) > eps:
        epochs += 1
        if adaptive:
            step = None # TODO: [6d] Implement adaptive learning rate
            ↪ update step

        for xt, yt in zip(X, y):
            guess = (yt - theta @ xt) * xt
            if guess.any():
                theta += learning_rate * guess
            n_iter += 1

        prev_loss = new_loss
        new_loss = calculate_squared_loss(X, y, theta)

    print("Learning rate:", learning_rate, "\t\t\tNum iterations:",
        ↪ n_iter)
    return theta
```

```
def ls_closed_form_optimization(X, y):
    """
    Implement the closed form solution for least squares regression.

    Args:
        X: np.array, shape (n, d)
        y: np.array, shape (n,)

    Returns:
        theta: np.array, shape (d,)
    """
    return np.linalg.inv(X.T @ X) @ X.T @ y
```

ii.

Algorithm	η	θ_0	θ_1	# iterations	Runtime (s)
GD	10^{-4}	0.276	0.175	117057	5.750
GD	10^{-3}	0.275	0.178	17326	0.851
GD	10^{-2}	0.275	0.179	2294	0.115
GD	10^{-1}	0.275	0.180	284	0.014
SGD	10^{-4}	0.275	0.179	188300	0.447
SGD	10^{-3}	0.274	0.180	19500	0.048
SGD	10^{-2}	0.267	0.186	5260	0.013
SGD	10^{-1}	0.220	0.253	780	0.003
Closed Form	–	0.275	0.180	–	0.007

(b)

Compare GD vs SGD: specifically, comment on runtime, number of iterations, and resulting coefficients at convergence

GD is more stable and needs fewer iterations, but each iteration is expensive. SGD is cheaper per iteration and much faster overall, but it is noisier and needs a smaller learning rate to approach the true coefficients. For this problem, GD with $\eta = 10^{-1}$ gives the best accuracy/runtime trade-off, while SGD with $\eta = 10^{-3}$ gives a close approximation at low runtime.

(c)

In this particular case, how does the runtime of the closed form solution compare to stochastic gradient descent? Which learning rate used for SGD produces the coefficients closest to the closed form solution?

The closed form solution is faster than SGD for learning rate above 10^{-1} . The learning rate that produces the best results for SGD is $\eta = 10^{-3}$ or $\eta = 10^{-4}$ since these both have one coord exactly and one coord off by 0.001.

(d)

```
if adaptive:
    step = 0.1/(1 + 0.1 * n_iter)
```

The coefficients are exactly the closed formula when rounded. It takes 38900 iterations to converge.

Problem 7: Logistic regression - Stochastic Gradient Descent

(a)

The logistic loss with respect to a single example $(\bar{x}^{(i)}, y^{(i)})$ is defined as

$$J^{(i)}(\bar{\theta}) = \ln(1 + e^{-y^{(i)}\bar{\theta} \cdot \bar{x}^{(i)}})$$

Find $\nabla_{\bar{\theta}} J^{(i)}(\bar{\theta})$.

$$\begin{aligned} \nabla_{\bar{\theta}} J^{(i)}(\bar{\theta}) &= \frac{1}{1 + e^{-y^{(i)}\bar{\theta} \cdot \bar{x}^{(i)}}} \left(\frac{dJ^{(i)}}{d\theta} 1 + e^{-y^{(i)}\bar{\theta} \cdot \bar{x}^{(i)}} \right) \\ &= \frac{-y^{(i)}x^{(i)}e^{-y^{(i)}\bar{\theta} \cdot \bar{x}^{(i)}}}{1 + e^{-y^{(i)}\bar{\theta} \cdot \bar{x}^{(i)}}} \\ &= \frac{-y^{(i)}x^{(i)}e^{-y^{(i)}\bar{\theta} \cdot \bar{x}^{(i)}}e^{y^{(i)}\bar{\theta} \cdot \bar{x}^{(i)}}}{\left(1 + e^{-y^{(i)}\bar{\theta} \cdot \bar{x}^{(i)}}\right)e^{y^{(i)}\bar{\theta} \cdot \bar{x}^{(i)}}} \\ &= \frac{-y^{(i)}x^{(i)}}{1 + e^{y^{(i)}\bar{\theta} \cdot \bar{x}^{(i)}}} \end{aligned}$$

(b)

$$\begin{aligned} \bar{\theta}^{(k+1)} &= \bar{\theta}^{(k)} - \eta \nabla_{\bar{\theta}} J^{(i)}(\bar{\theta}^{(k)}) \\ &= \bar{\theta}^{(k)} + \eta \frac{y^{(i)}x^{(i)}}{1 + e^{y^{(i)}\bar{\theta} \cdot \bar{x}^{(i)}}} \\ &= \bar{\theta}^{(k)} + \eta y^{(i)}x^{(i)} \sigma(-y^{(i)}\bar{\theta} \cdot \bar{x}^{(i)}) \end{aligned}$$

(c)

```
def sigmoid(z):
    """
    Implements the sigmoid function..
    Args:
        z: A scalar or numpy array of any size
    """
    return 1 / (1 + np.exp(-z))
```

```
def logistic_stochastic_gradient_descent(X, y, lr=0.0001):
    """
    Implements the Stochastic Gradient Descent (SGD) algorithm for logistic
    ↪ regression.
    Note:
        - Please do not shuffle your data points.
        - Please use the stopping criteria: number of epochs == 10,000

    Args:
        X: np.array, shape (n, d)
        y: np.array, shape (n,)
        lr: the learning rate for the algorithm

    Returns:
        theta: np.array, shape (d+1,) including the offset term.
    """
    n, d = X.shape
    theta = np.zeros(d + 1)

    epochs = 10000
    for epoch in range(epochs):
        for Xt, yt in zip(X, y):
            xt = np.insert(Xt, 0, 1.0)
            theta = theta + lr * yt * xt * sigmoid(-yt * (xt @ theta))
    return theta
```

$$\bar{\theta} = \begin{bmatrix} 0.180 \\ -0.011 \\ 0.055 \end{bmatrix}$$

(d)

Can you find a closed form solution for logistic regression? If so, derive it. If not, explain why.

No you find a closed solution. This is because when we take the gradient and set it to zero we get a sum of $\nabla_{\bar{\theta}} J^{(i)}$. The problem is a sum of sigmoid functions cannot be solved algebraically.